

From FHIR to Graph

Healthcare Analytics on Parquet + PuppyGraph

Synthea FHIR | DuckDB | Parquet | Graph Traversal

Puppygraph use case demo | Seattle | 2026

The Pipeline: FHIR to Graph in 4 Steps

1

Synthea FHIR JSON

Synthetic patient records as FHIR Bundles. Each bundle contains Patient + Conditions + Observations linked by UUID references.

2

Parse + Flatten

Python resolves UUID graph references into explicit patient_id foreign keys. Output: 3 clean DataFrames.

3

Parquet via DuckDB

DataFrames written to Parquet. DuckDB materializes as actual tables (no path references) and is copied into the Docker container.

4

PuppyGraph Schema

schema.json maps tables to graph. patients = vertices, conditions rows = both vertices AND edges via patient_id foreign key.

Same Question. Very Different Queries.

"Find all patients with Prediabetes AND Obesity, then show their other conditions"

SQL | 3 Self-Joins

```
SELECT c3.condition_display, COUNT(*)
FROM patients p
JOIN conditions c1 ON p.patient_id = c1.patient_id
JOIN conditions c2 ON p.patient_id = c2.patient_id
JOIN conditions c3 ON p.patient_id = c3.patient_id
WHERE c1.condition_display LIKE '%Prediabetes%'
      AND c2.condition_display LIKE '%obesity%'
      AND c3.condition_display NOT LIKE '%Prediabetes%'
GROUP BY c3.condition_display
ORDER BY count DESC LIMIT 10
```

Gremlin | One Traversal

```
g.V().hasLabel('Condition')
    .has('condition_display',
        containing('Prediabetes'))
    .in('HAS_CONDITION')
    .where(
        __.out('HAS_CONDITION')
            .has('condition_display',
                containing('obesity')))
    .out('HAS_CONDITION')
    .values('condition_display')
    .groupCount()
    .order(local).by(values, desc)
    .limit(local, 10)
```

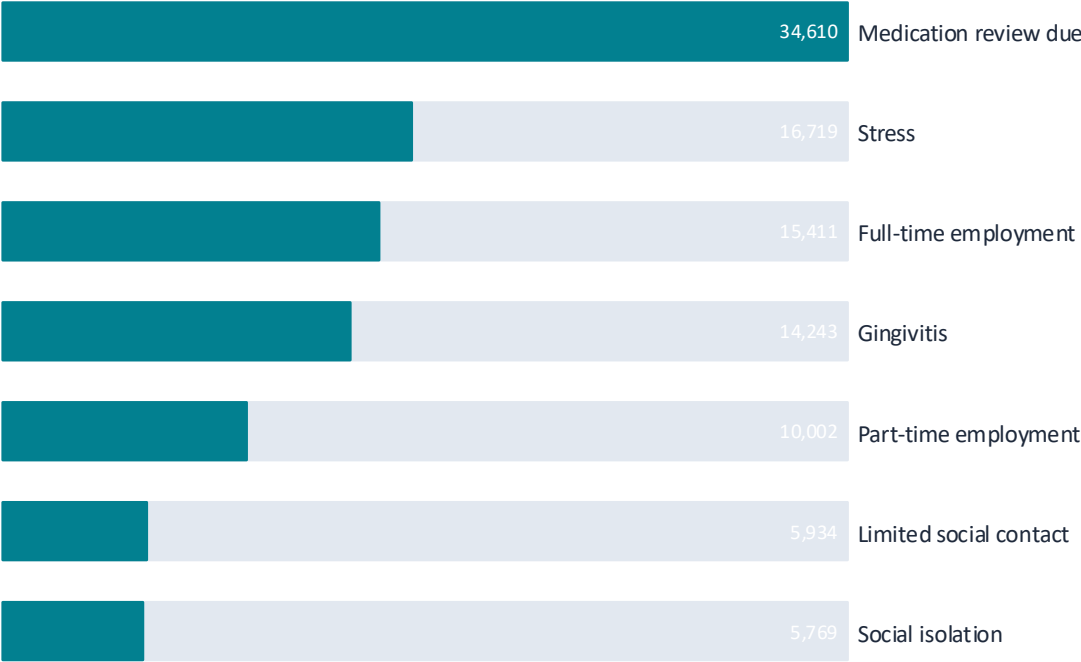
What the Graph Found

Top Comorbidities

3,881

patients with both
Prediabetes + Obesity

Query time: 0.922s



Why Graph on Parquet?

No new database

PuppyGraph reads your Parquet files directly via DuckDB. No ETL into a graph DB. No data duplication.

Multi-hop queries in one traversal

Patient > Condition > Observation chains that would need 3+ SQL JOINS are a single Gremlin walk. SQL complexity grows with depth. Gremlin does not.

SQL overcount problem

Three self-joins on conditions produce cartesian row explosions. 20 conditions per patient = 8,000 intermediate rows before filtering.

Natural fit for FHIR data

FHIR is already a graph: Patient references Condition references Observation. Parquet preserves the foreign keys. PuppyGraph exposes them as edges.